DOMAIN 1 · PLAN & CONFIGURE

Use cases first, agents second.

A **use case** is a recurring business outcome a named person can describe, time-box, and tell you when it succeeded — observed before any agent is named.

Use cases classify into three fit zones. Naming the zone is the first planning decision; everything downstream — pattern, environment, DLP, governance — depends on getting it right.

THREE FIT ZONES

ZONE A

Judgement call

High demand, medium clarity. Sources exist; answers vary. A human needs the loop kept open.

Lean into

Pilot narrow, measure deflection, design escape hatches for the boundary cases.

ZONE B

Clear fit

High demand, high clarity. Same question, same source, asked > 50× a week.

Lean into

Templates first; let patterns emerge; index success on deflection, not on satisfaction.

ZONE C

Avoid

High-stakes, novel, or rare. Bad data, ambiguous facts, life-or-money decisions.

Lean into

Out of scope. Keep humans in the seat. If you must, fence narrowly and never let it publish.

PATTERNS · A CATALOGUE OF SHAPES

Five shapes most agents will eventually become.

A **design pattern** is a tested shape — instructions, knowledge, actions, and orchestration — that solves a recurring class of problems. Pick the shape that fits the use case, not the one you just watched a video about.

PATTERN 01

Q&A assistant

Single-source retrieval. The user asks, the agent finds, the agent cites.

Use when

A trust center, a policy library, a 100-page handbook.

Failure mode

Stale index, over-long answers, no escalation.

PATTERN 02

Triage / routing

Reads intent, picks the right human or system, hands context along.

Use when

Service desk fan-out, IT helpdesk, ticket sorting.

Failure mode

Crooked taxonomy; rewrites user intent too aggressively.

PATTERN 03

Process agent

Multi-step orchestrated task across systems, ends with a deliverable.

Use when

Onboarding, vendor setup, multi-system case work.

Failure mode

Halfway done is worse than not started — plan exit conditions.

PATTERN 04

Supervisor

A parent agent that delegates to specialised children and stitches results.

Use when

Multi-domain answers; one agent cannot hold the whole surface.

Failure mode

Loops forever; rename responsibilities before you ship.

PATTERN 05

Autonomous

Event-triggered, multi-step, observes & reacts, escalates.

Use when

Long-running processes an SRE would otherwise babysit.

Failure mode

Spends credits without shipping value. Needs observability first.

CONVERSATIONAL DESIGN · SIX RULES

Conversations are designed, not improvised.

Conversational design is the discipline of shaping every turn — its shape, its silence, its hand-off — so that a user lands where they meant to go without ever meeting the model.

PRINCIPLE 01

One job per turn

A turn carries one intent and resolves it. The next turn can carry another. Bundling turns into long prompts is a coin you spend twice.

PRINCIPLE 02

Confirm before acting out

Anything that writes, sends, files, deletes, or charges should pause for a yes. Read-only is silent. Write is gated.

PRINCIPLE 03

Cite the evidence

Every claim that came from a source attaches to that source. Trust is a citation, not a tone of voice.

PRINCIPLE 04

Hand off cleanly

Define what the agent does when it doesn't know. Handoff is a destination, not an apology.

PRINCIPLE 05

Design for silence

Pause, don't panic. Re-state next step, don't repeat yourself. Silence is a feature; chatter is a bug.

PRINCIPLE 06

Compose with constraints

Budget tokens. Bound tools. Bind styles. A constrained turn reads better than a free one — and ships faster.

CONVERSATIONAL DESIGN · ANATOMY OF A TURN

One turn, end to end.
Eight steps. None optional.

A **turn** is the unit the user feels. Underneath the surface it is a chain of decisions, each costumerable, each observable. Plan the chain first; tune the words second.



COST MAP

Stages 04 + 06 spend money — token cost in 04, action cost in 06. Stages 01, 02, 07, 08 set the floor. 03, 05 spend the least and save the most.

TO REMEMBER

If the same turn covers more than one user job, split the turn. If a stage can be skipped safely, skip it. If a stage can stop the agent, give it teeth.

PERSONAS & SCOPE · MAP WHO, WHERE, WHAT

Same agent, different scopes — different agents.

A **persona** describes who shows up on the other side. **Scope** describes what surfaces, tools, and knowledge they reach. Pair them on purpose — an unsplit persona-times-scope matrix creates accidental power.

THREE SAMPLES

Persona	Scope	
PERSONA A **Tier-1 support rep**	SCOPE **Read 4 knowledge bases, open 2 internal systems** No write. No external data. Confirms silence, never pesters.	read
PERSONA B **HR partner reviewer**	SCOPE **A draft, a checklist, a recommendation** Write into review queue. Always confirms before send.	draft
PERSONA C **Vendor ops lead**	SCOPE **Create vendor records, post updates, file tickets** Writes are gated. Acknowledged in audit. Mistakes show up in a queue.	act

Write the persona, name the scope, stick them on a wall. Re-decide on purpose when either one moves.

THREE RULES

Pair persona to scope on purpose, then name it.

If a Tier-1 rep needs to file a record, hand them a different agent. Don't grow the Tier-1 agent's scope to make one person happy.

- **Same persona, different scope — split.**
- **Same scope, different persona — branch channels.**
- **Both shift — retire, build, replace.**

ENVIRONMENT STRATEGY · WHERE WORK HAPPENS

An environment is a postcode for change.

The strategy answers: who can change what, where, and when. Choose a topology first — then the controls. Topology decides whether your team can move at all.

01 · MAKER-OWNED

Developer (sandbox)

1 maker · quick reset · throwaway · free of policies

02 · TEAM-MERGED

Build

A few makers · solution checked in · branches merge here

03 · SHARED ISOLATED

Test / UAT

Validates the solution · synthetic data · real perms

04 · LOCKED-DOWN

Production

Real traffic only · DLP strict · telemetry on · managed + observed

05 · A FLAG, NOT A FOURTH TYPE

Managed Environments wrap any of the above.

Adds limits, sharing controls, usage insights, weekly digests, the lot. Enable on prod first; on Test next.

DECISIONS YOU'LL BE ASKED

A topology that fits your team.

- One development topology per maker team. Not per business unit.
- Region is decided once, kept with the workload, and re-decided rarely.
- Test gets a copy of production data shape, never the data itself.
- First Managed Environments always go on production.

DEFAULT PER ENVIRONMENT

15 GB database · 20 GB file · 2 GB log

Adjustable in the Power Platform admin center.

DLP · THREE DATA GROUPS

DLP is classification, not a firewall.

A **data-loss-prevention (DLP) policy** classifies every connector into one of three buckets — and stops data from crossing between them. The buckets decide what an agent may know, may write, and may touch.

BUCKET 01
Business data

Connectors allowed to share business data freely.

Example: Dataverse, SharePoint, approved first-party.

Use for anything that holds, reads, or writes data an employee would consider sensitive.

BUCKET 02
Non-business

Connectors confirmed to be not for business data.

Many tenants block these by default.

Useful for personal productivity. Dangerous for an agent that touches customer data.

BUCKET 03
Blocked

Connectors with no business purpose.

Publishing an agent that uses a Blocked connector is denied.

Treat as a deny-list, not a suggestion.

THE RULE EVERY POLICY MUST OBEY

A connector may share data only with connectors in the same group. Cross-group data flow is denied by the runtime — not by review, not by alert: denied.

Policies are scoped to environments and applied non-destructively — a connector classified one way in Dev can be classified another in Prod.

DLP · DESIGNING THE DECISION TREE

For each connector your agent calls, ask five questions, in order.

Use this decision tree every time you introduce a connector to an agent. Most "weird" outages come from skipping a step here.

Q1
Does the agent need it?
No → remove. Yes → continue.

→

Q2
What data does it touch?
Customer, HR, finance, public, none?

→

Q3
Will it read or write?
Write changes the size of risk.

→

Q4
Whose credential?
User, service principal, sharing?

→

Q5
Place it.
Pick a group. Document.

THREE POSSIBLE OUTCOMES

OUTCOME A
Place in Business
It holds or touches business data. May share with other Business connectors. Pair it with a connection reference. Document who owns the credential.

`Default for life-of-the-business connectors.`

OUTCOME B
Place in Non-business
Personal productivity, social surfaces, anything an end user would never use to share business data. Tenant default may block these; choose explicitly.

`Loud warning when used by a customer-facing agent.`

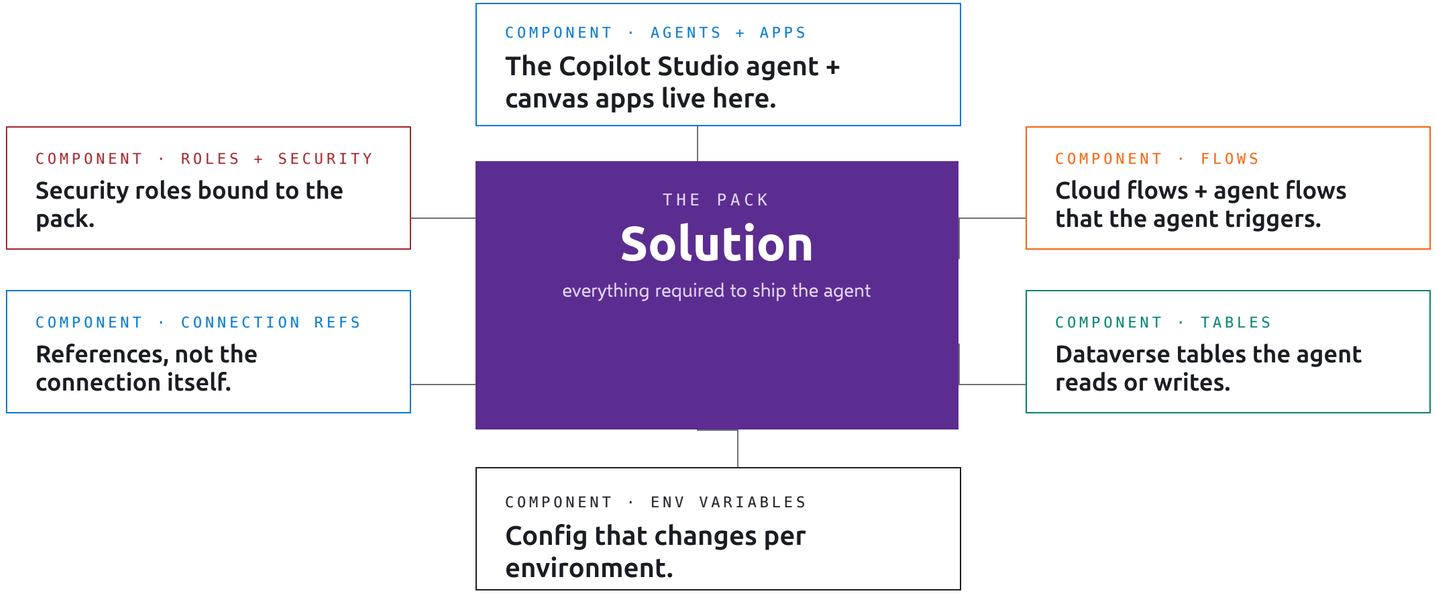
OUTCOME C
Place in Blocked
No business purpose — risky or competitive connectors. Any agent that tries to publish with one is denied. Treat as deny-list, not suggestion.

`A policy that uses this list stays loud.`

SOLUTIONS · THE UNIT OF PROMOTION

A solution is a portable backpack that travels between environments.

A **solution** is a pack that captures the artefact, its schema, its references, and its metadata. You author in **unmanaged** form in Dev; you import and run as **managed** in Test and Prod.



THREE HABITS

- **One solution, one purpose.**
- **Solve layer interactions in their own pack.**
- **Promote by export, never by re-clone.**

MANAGED VS UNMANAGED

Unmanaged in Dev means "I'm authoring this, you can edit". Managed in Test and Prod means "the owner of this is the source — you import to consume". Once installed as managed, an import-update cannot delete components; only add or extend.

SOLUTION & SECURITY GOVERNANCE

Solutions live inside layers that hold them accountable.

Governance is the layered system that decides who can install, who can edit, who can publish, and who reviews what shipped. Each layer has its own owner's name, dates, and rules.

LAYER A

Tenant policy

Default DLP · default settings · maker-onboarding controls · audit Search.

LAYER B

Environment governance

Env-scoped DLP · backups · Managed Environments toggles · capacity pinning.

LAYER C

Solution-level governance

Publisher · owner · description · version · required/optional dependencies · security roles.

FOUR CONTROLS THAT TRAVEL WITH EVERY SOLUTION

01 · WHO CAN INSTALL

Install permission bound to a security role on each environment.

02 · WHO CAN EDIT IN DEV

Solution Checker + Maker permission · co-author · reviews.

03 · WHO CAN PUBLISH

Sharing, channel-by-channel · security groups named with intent.

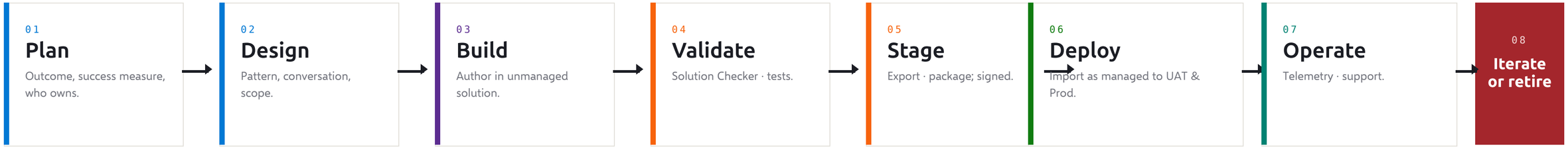
04 · WHO REVIEWS WHAT SHIPPED

Audit log + change history · approvals + signoff owners.

ALM · LIFE OF AN AGENT SOLUTION

ALM is eight stages — and one loop you do not skip.

Application lifecycle management (ALM) is the discipline that takes an outcome from idea to retirement — and through every change in between. Stages repeat; decisions don't.



3 HABITS

— Stage 03 + 04 are where quality lives.

Validate twice. Solution Checker first, testers second. The cost of a missed issue is exponential — Dev-cost, UAT-cost, Prod-incident cost.

— Stage 07 is the loop, not the finish line.

Telemetry is the input to Stage 03 again. Operate that doesn't feed Plan is decoration. The cycle closes or the work rots.

— Stage 08 is honest, not lazy.

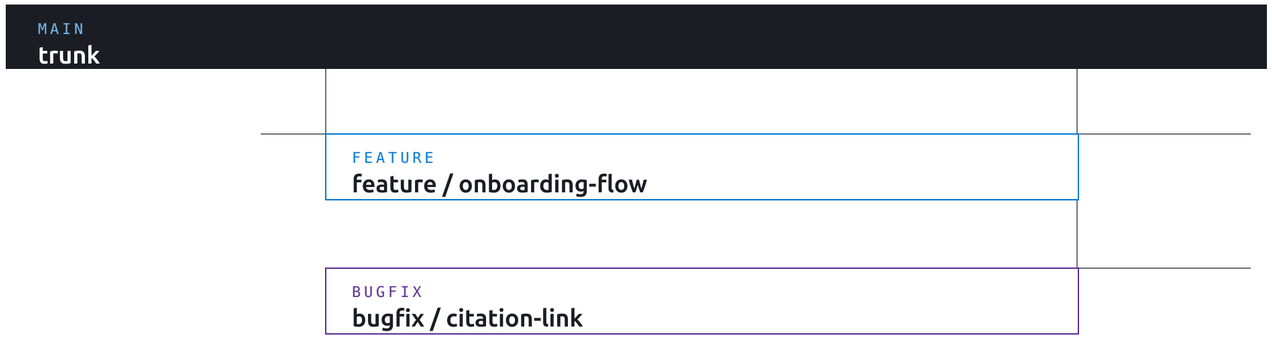
Retiring is the work of removing the agent from channels, archiving the solution, and keeping the audit log. Don't end with a ZOMBIE.

BRANCHING & PROMOTION

A branch is a promise — an environment is a promise kept.

A **branch** is a working lane where work-in-progress lives. An **environment** is the place that lane ultimately serves. **Promotion** is the act of carrying a finished lane into its environment.

BRANCHING



THREE HABITS OF A HEALTHY BRANCH

Short. Single-purpose. Named.

Two-week-old lanes cost more than two-week-old code. Branch hygiene keeps the team moving.

PROMOTION

The lane meets its environment.

01
Export the unmanaged solution from Dev

02
Import as managed to Test — validate, sign off

03
Re-import to Prod — same managed solution

WATCH FOR

Once installed as managed, updates cannot remove

Components only add or extend. Plan additions; track retirements in a separate, named solution.

RULE

A re-import closes the loop — it never replaces the original.

CONNECTIONS · WHERE DATA CROSSES

A connection is a credential wrapped in a connector.

A **connector** is the API surface your agent talks to. A **connection** is one credentialed setup of that connector bound to a user or a service principal. Knowing the difference is half of doing this well.

01 · STANDARD

Standard connectors

Built-in, no extra licence, included in the platform.

Common surface for Office 365, SharePoint, Outlook.

02 · PREMIUM

Premium connectors

Licenced per-call or per-tenant. Third-party or specialised.

Adobe, Salesforce, ServiceNow, HTTP webhooks.

03 · CUSTOM

Custom connectors

Your own OpenAPI / Postman collection becomes a connector.

Owned by the maker who registered them.

04 · VIRTUAL

Virtual connectors

A wrap that points at an existing connection.

Useful for DLP-staged boundaries.

A CONNECTION, BROKEN DOWN

LAYER 01 · API
Connector (the surface)

LAYER 02 · AUTH
Identity & consent

LAYER 03 · SCOPE
Who is allowed to use it

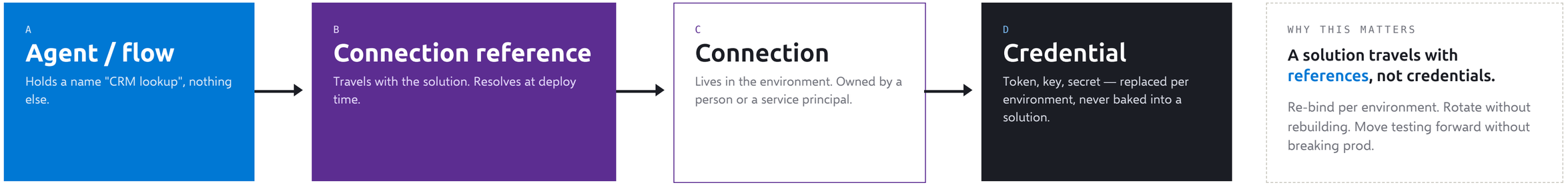
LAYER 04 · POSTURE
DLP & audit

A connection without a credential is a stub. A credential without consent is a leak. A connection without DLP is the loudest outage you will ever debug.

CONNECTION REFERENCES

A reference, not the thing. That difference saves solutions.

A **connection reference** is a solution component that points at a real connection. The reference travels; the connection stays behind. The reference is what gets bound to the agent; the connection is what holds the credential.



WHY THIS MATTERS

A solution travels with references, not credentials.

Re-bind per environment. Rotate without rebuilding. Move testing forward without breaking prod.

WHAT LIVES WHERE

Travelling

The agent, the agent flows, env variables, the connection references themselves.

IN THE SOLUTION

Per environment

The connection itself — Test gets its own; Prod gets its own. Identity is whoever owns the env's secret store.

BOUND AT DEPLOY

Replaceable

When a credential rotates, only the connection object changes. The reference and the agent do not.

ROTATION-FRIENDLY

Shared via SP

Connections can be shared only with service principals — not with individuals. Plan for it.

GOVERNANCE NOTE

LAB · PLAN & CONFIGURE

Design the solution blueprint, configure the environment.

Ten steps. Five are paperwork, five are design. The paperwork halves the design's risk — skip it and the surprise is on you.

01 OUTCOME
Identify the outcome & success measure
One sentence. With a metric. With a deadline.

02 PATTERN
Map use case to a pattern
Q&A, triage, process, supervisor, autonomous.

03 PERSONAS
Sketch personas & scope
Who shows up. What they may touch. What they may not.

04 CONVERSATION
Sketch the turn
Trigger, branches, hand-offs. Cost per stage named.

05 TOPOLOGY
Plan environments & users
Dev · Build · Test · Prod. Region. Managed Env on Prod first.

06 SOLUTION
Sketch the solution components
Agent, flows, tables, refs, env vars. One pack, one purpose.

07 DLP
Classify each connector
Five-question tree per connector. Document the choice.

08 REFERENCES
Add connection references
References in the solution — never the connection itself.

BY THE END

A blueprint is something you can hand to a peer.

- — One outcome, named & measured.
- — Pattern + persona-to-scope map.
- — Environment topology drawn.
- — Solution components listed.
- — DLP decisions recorded.
- — Connection references added.
- — A peer can refuse.

DO

Hand to a peer before authorising. The best blueprint is one someone else can refuse.